Concept guide

Designing mixed cryoprotectants with solid–liquid equilibrium

Siddhant Chowdhary 25B0347 CL207 optional project

The whole project in five lines

1. When a solution freezes, ice is pure water, so everything dissolved gets squeezed into the shrinking liquid. For a frozen cell, that concentrated liquid is a major source of damage.
2. Thermodynamics predicts exactly how concentrated it gets at each temperature — solid–liquid equilibrium plus an activity-coefficient correction.
3. The correction is written *per –OH group*, so one number fitted on glycerol can predict propylene glycol, sorbitol, and mixtures of them.
4. We test that prediction blind, then use the model to design the least-toxic glycerol + sorbitol recipe that keeps salt in the leftover liquid below a target.
5. The pieces are known (mostly from food science); using them together to design a cryoprotectant mixture is the new part.



Figure 1: The path from data to answer. Each box is a section of this guide.

How to read this guide. Each concept follows the same pattern: *what it is* in one line, *why* the project needs it, *how* it works, *where* it lives in the code, and a one-sentence takeaway. Part A is the physics, Part B the measurement, Part C how data become an answer.

Part A — The physics

1 Freezing and freeze concentration

Ice is almost perfectly pure water, so freezing concentrates everything left behind.

Why we need it. This is the mechanism the whole project is about. A cell in slow freezing is not mainly destroyed by ice crystals touching it; it sits in a liquid that grows steadily saltier, which pulls water out of the cell and exposes it to damaging concentrations. Cryobiologists call this *solution effects injury*.

How it works. Water molecules fit into the ice lattice; solute molecules do not. As ice grows, the same amount of solute is shared by less and less water. Because solute is conserved, one line of arithmetic gives the leftover liquid:

$$\frac{m_{\text{liquid}}(T)}{m_0} = \frac{w_0}{w(T)}$$

where w_0 is the starting solute mass fraction and $w(T)$ the fraction in the liquid at T. Thermodynamics supplies $w(T)$; the rest is bookkeeping.

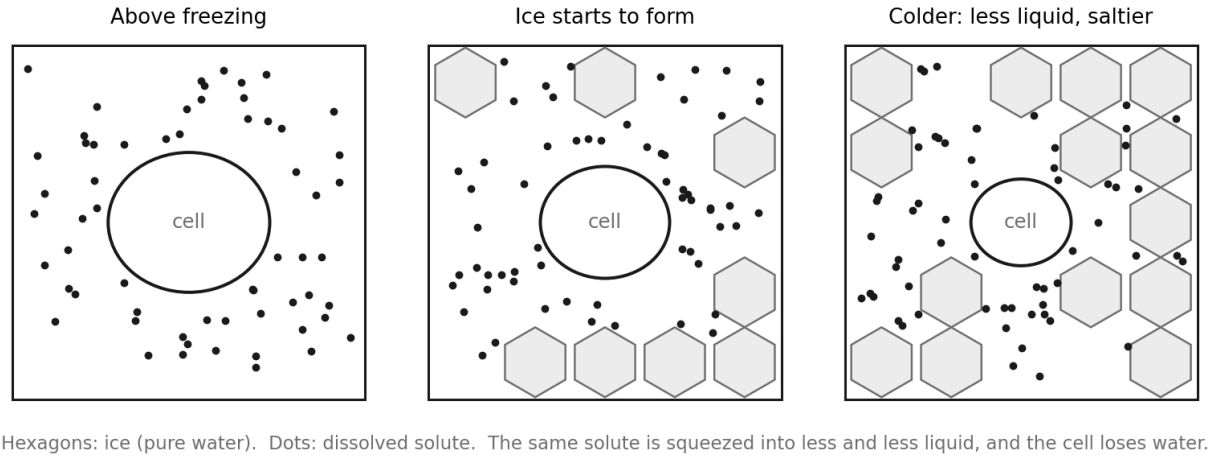


Figure 2: Freeze concentration. The number of solute dots is the same in every panel.

In the code. `thermo/sle.py` → `residual_liquid()`

Takeaway. Predicting $w(T)$ is predicting what a frozen cell actually experiences.

2 Equilibrium: why "equal fugacity"

Two phases stop exchanging a molecule when that molecule's "escaping tendency" is the same in both.

Why we need it. We need a condition that tells us *at which temperature* ice and a given solution can sit side by side. That condition is what turns into the freezing-point equation.

How it works. Intuitively: water molecules constantly leave the ice and join it. If they leave faster than they join, the ice melts; if slower, it grows. At equilibrium the rates balance. The thermodynamic quantity that measures a molecule's tendency to leave a phase is its chemical potential μ, or equivalently its fugacity f (Lectures 15–17). So the condition is

$$\bar{f}_w^{\text{liquid}}(T, P, x) = f_w^{\text{ice}}(T, P).$$

The ice side has no composition in it because ice is pure water. All of the solution's influence enters through the liquid side.

Takeaway. Equal fugacity is the single physical statement everything else is built on.

3 The solid–liquid equilibrium equation

Rearranging "equal fugacity" gives the freezing temperature for any composition.

Why we need it. This is the working equation of the project — every predicted freezing point, every leftover-liquid calculation and every recipe comes out of it.

How it works. Write the liquid fugacity relative to pure liquid water, $\bar{f}_w = x_w \gamma_w f_w^L$. Then

$$\ln(x_w \gamma_w) = \ln \frac{f_w^{\text{ice}}}{f_w^L}.$$

Below 0 °C pure liquid water is hypothetical, so its fugacity is reached by a three-step path (Lecture 25): heat the ice to its melting point, melt it, cool the liquid back. That gives the free energy of

fusion, and with a constant enthalpy of fusion,

$$\boxed{\ln(x_w\gamma_w) = -\frac{\Delta_{\text{fus}}H}{R}\left(\frac{1}{T} - \frac{1}{T_m}\right)} \tag{1}$$

Read it term by term:

- $x_w\gamma_w = a_w$ is the *water activity* — how "available" the water in the solution is. Solutes lower it.
- The right side depends on temperature only. Lower T, more negative right side.
- So a solution with lower water activity must be colder before ice can form. That is freezing-point depression, derived rather than memorised.

A refinement adds the heat-capacity difference between liquid water and ice, $\Delta C_P \approx 38\,\text{J mol}^{-1}\,\text{K}^{-1}$. The project *tests* whether this matters over its temperature range rather than assuming it does not.

In the code. `thermo/sle.py` → `freezing_point()`, `ln_aw_ice()`

Takeaway. One equation, one idea: less available water means a lower freezing point.

4 Ideal solutions — and why they fail here

An ideal solution assumes the molecules do not care who their neighbours are.

Why we need it. It is the baseline. Set $\gamma_w = 1$ in equation (1) and you get the textbook freezing-point depression. We need to know how far reality sits from it.

How it works. In an ideal solution water's activity equals its mole fraction, $a_w = x_w$. That works when solute molecules interact with water about as strongly as water does with itself. Polyols do not: each –OH group hydrogen-bonds to water, so water "prefers" being next to them. The ideal model starts drifting as concentration rises — and the concentrations that matter in freeze concentration are exactly the high ones.

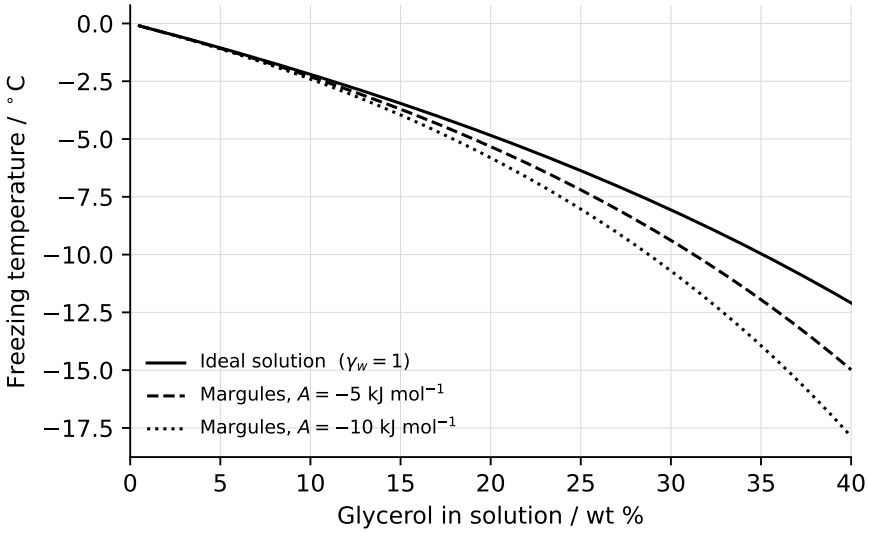


Figure 3: Ideal versus non-ideal freezing curves for glycerol–water. The gap grows with concentration; below about 10 wt% the curves almost coincide.

Takeaway. The ideal model is right at low concentration and wrong precisely where it matters.

5 Activity coefficients and excess Gibbs energy

The activity coefficient γ is the correction factor that turns an ideal solution into a real one.

Why we need it. Without it the model is only the ideal baseline. With it, it can match real data.

How it works. Rather than guessing γ directly, we write a model for the *excess Gibbs energy* G^{ex} — the part of the mixing energy that an ideal solution misses — and differentiate it (Lectures 21–25). The simplest model that behaves correctly is one-constant Margules:

$$\frac{G^{\text{ex}}}{RT} = \alpha\, x_s x_w, \qquad \alpha = \frac{A}{RT} \quad \Longrightarrow \quad RT \ln \gamma_w = A\, x_s^2.$$

A negative A means water and solute attract each other more than an ideal mixture would, so water is *less* available ($\gamma_w < 1$) and the freezing point drops further.

Building from G^{ex} matters for one reason: it guarantees the **Gibbs–Duhem** relation, $x_w\, d\ln\gamma_w + x_s\, d\ln\gamma_s = 0$. The activities of all components are linked, and a model that breaks this link is thermodynamically inconsistent. The code checks it numerically (the residual comes out around 10^{-14}).

In the code. `thermo/activity.py` → `MargulesModel`; `thermo/fit.py` → `gibbs_duhem_residual()`

Takeaway. γ is where the chemistry lives; deriving it from G^{ex} keeps it honest.

6 Group contribution: one number per –OH group

Instead of one parameter per molecule, give each –OH group its share of the interaction.

Why we need it. A Margules constant fitted on glycerol describes *glycerol* and water. It says nothing about sorbitol. To predict a molecule you have not measured, the parameter has to belong to a *piece* of the molecule that different molecules share.

How it works. In polyols the dominant interaction with water is hydrogen bonding through –OH groups. So we set

$$A_i = n_{\text{OH},i}\, a_{\text{OH}},$$

with one parameter a_{OH} shared by every polyol. Fit it on glycerol (3 –OH); predict propylene glycol (2) and sorbitol (6). This is a stripped-down version of how the UNIFAC method works, and UNIFAC has already been applied to polyol solutions in food science — which is why this step is cited, not claimed.

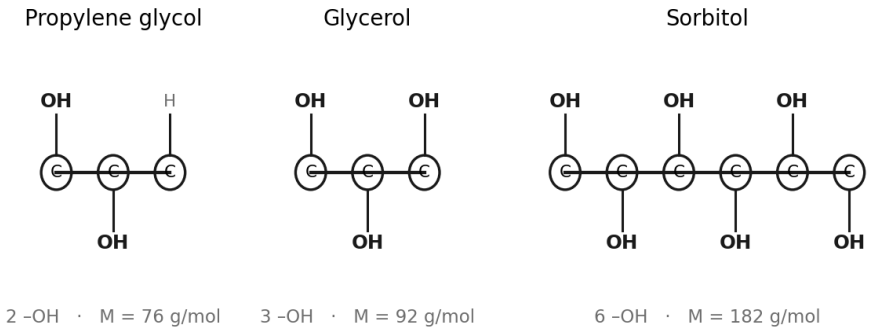


Figure 4: The polyol series. The model treats each –OH as an equal contributor.

The assumption being tested is that every –OH counts the same, wherever it sits and whatever the molecule's size. If sorbitol misses badly, that assumption — or molecular size — is the likely culprit.

In the code. `thermo/activity.py` → `GroupContributionModel`

Takeaway. Parameters that belong to parts of molecules are what make prediction possible.

7 Mixtures, with no new parameters

The same per-group parameter handles several solutes at once.

Why we need it. The recipe is a mixture, so the model must predict mixtures — ideally without fitting anything new, so that the mixture measurement is a genuine test.

How it works. With only water–solute interactions kept,

$$\frac{G^{\mathrm{ex}}}{RT} = \sum_i \alpha_i x_i x_w \quad \Longrightarrow \quad \ln\gamma_w = (1-x_w)\sum_i \alpha_i x_i, \qquad \alpha_i = \frac{n_{\mathrm{OH},i}\, a_{\mathrm{OH}}}{RT}.$$

For a single solute this collapses back to Margules exactly. Solute–solute interactions are neglected — a stated assumption, and the mixture measurements test it.

Takeaway. A blind mixture prediction is the strongest test the model can face.

8 The key insight: temperature alone fixes the water activity

Once ice is present, the liquid's water activity is set by temperature — not by the cryoprotectant.

Why we need it. This changes what "better cryoprotection" can even mean, and it is the reason the recipe is designed the way it is (Section 13).

How it works. Look again at equation (1): its right side contains only T. Any liquid in contact with ice at $-10\,^{\circ}\mathrm{C}$ has the same water activity, whatever is dissolved in it. So its osmotic pressure is also fixed:

$$\Pi = -\frac{RT}{\bar{V}_w}\ln a_w \approx 12\ \mathrm{MPa}\ \text{at}\ -10\,^{\circ}\mathrm{C} \quad (\text{about } 120\ \mathrm{bar}).$$

That is the force pulling water out of a cell at that temperature, and no cryoprotectant changes it. What a cryoprotectant *can* change is how much liquid is left and *what that liquid contains* — in particular, how diluted the salt from the cell medium is.

In the code. `thermo/sle.py` → `osmotic_pressure_at_ice()`

Takeaway. Cryoprotectants cannot change the osmotic stress at a given temperature; they change the company the cell keeps.

Part B — The measurement

9 Reading a cooling curve

Log temperature against time while a sample freezes; the freezing point is hiding in the shape.

Why we need it. This is how each experimental freezing point is obtained.

How it works. Five features, in order:

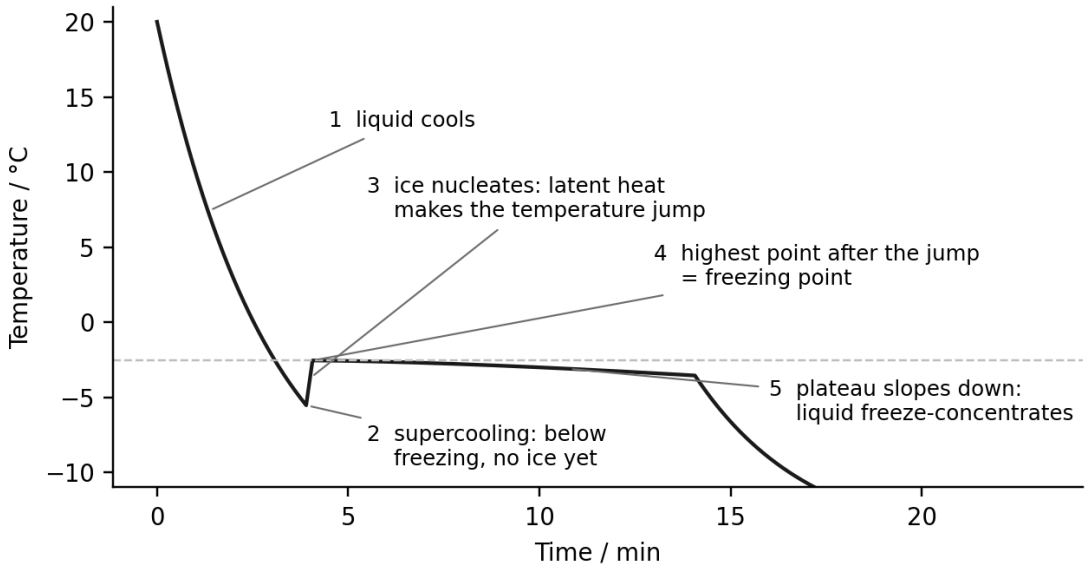


Figure 5: Anatomy of a cooling curve for a solution.

Two details decide whether the number is right. First, the dip (supercooling) is *not* the freezing point — ice had not formed yet. Second, a solution's plateau slopes downward because the liquid freeze-concentrates as it goes. So the best estimate is the *highest* point just after the jump. Seeding a tiny ice crystal when the sample is about 1 °C below its expected freezing point shortens the dip and gives a cleaner curve.

In the code. `analysis/cooling_curve.py` → `find_freezing_point()`; `analysis/simulate_curve.py` tests it on fake curves

Takeaway. Log the whole curve and take the peak after the jump, never the first dip.

10 The rig and calibration

Three probes, three tubes, one ice–salt bath, one Arduino.

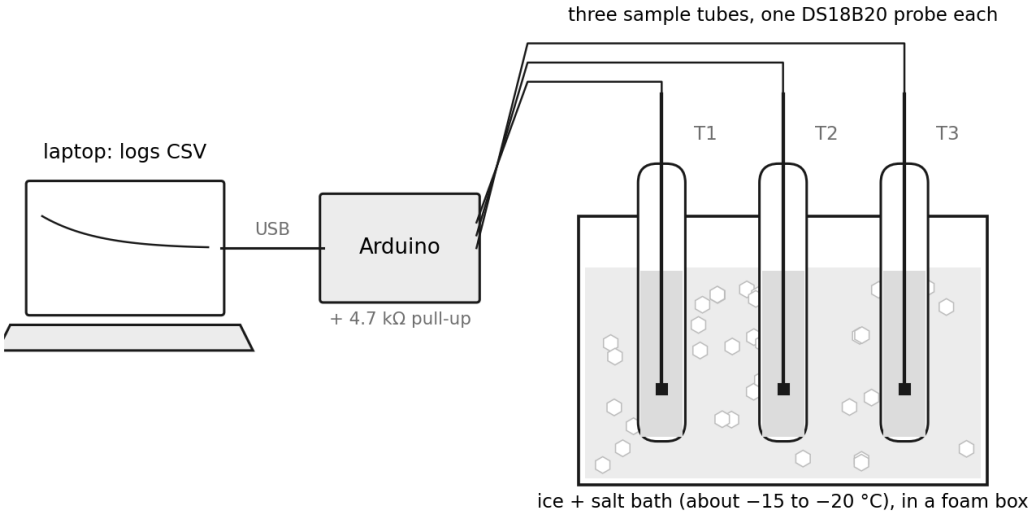


Figure 6: Experimental setup.

Why we need it. Cheap probes have a resolution of 0.06 °C but an absolute accuracy of only ±0.5 °C. Without calibration, every number would carry an unknown offset.

How it works. Run pure water first: it must read 0 °C, and whatever it actually reads is the probe's offset, subtracted from every later run. Then run a sodium chloride solution whose freezing point is tabulated, as an independent check. Measure each point twice (or three times) so every freezing point has an error bar.

In the code. `arduino/ds18b20_logger.ino; arduino/serial_to_csv.py`

Takeaway. Calibration turns a cheap probe into a trustworthy one.

Part C — From data to answer

11 Fitting and uncertainty

Choose a_{OH} so predicted freezing points match measured ones as closely as possible — and say how sure you are.

Why we need it. A parameter without an uncertainty cannot be tested: you cannot say whether a prediction "missed" without knowing how wide the prediction's band is.

How it works. *Least squares* picks the a_{OH} that minimises the sum of squared differences between measured and predicted freezing points. *Bootstrapping* then estimates the uncertainty: resample the data points with replacement hundreds of times, refit each time, and read the spread of the results.

One warning shows up immediately. At low concentration x_s^2 is tiny, so a small error in a measured freezing point produces a large error in A — a 0.2 °C error at 10 wt% glycerol shifts A by roughly 10 kJ mol^{-1}. That is why the fit uses many concentrations, not one, and why the higher concentrations carry most of the information.

In the code. `thermo/fit.py` → `fit_group_contribution(), bootstrap()`

Takeaway. Fit across the whole range, and always quote the error bar.

12 The blind prediction

Make the prediction, lock it in, and only then look at the answer.

Why we need it. A model fitted to data will always match that data; that proves little. It earns trust only by predicting something it has not seen.

How it works. Fit on glycerol only. Predict propylene glycol, sorbitol and the mixtures. **Commit the predictions to git before measuring.** The commit timestamp is independent proof that the prediction came first. Then compare. If it misses, the pattern of the miss is diagnostic:

Pattern of the error	Likely cause
Grows with molecular size (worst for sorbitol)	Size matters, not just –OH count → Wilson model
Scattered, mostly at low concentration	Measurement noise
Only at high concentration	Approaching the glass-forming region
Consistent offset in one direction	–OH groups are not all equivalent

Takeaway. A prediction that could have failed and did not is worth more than a perfect fit.

13 Designing the recipe

Find the least-toxic glycerol + sorbitol recipe that still protects the cell.

Why we need it. Real protocols mix cryoprotectants because each becomes toxic at high concentration. Splitting the dose lowers each agent's exposure. The model should be able to say *how* to split it.

How it works. The first formulation we tried — fix the total dose, then minimise the worst normalised toxicity — turned out to be empty: its best ratio is simply the ratio of the two toxicity limits, whatever the thermodynamics. (Both solutes are conserved in the leftover liquid, so their ratio never changes.) That finding is itself worth one sentence in the report.

The working formulation lets both doses vary and demands a thermodynamic protection target:

- **Target:** at the storage temperature, the salt concentration in the leftover liquid must stay below a chosen limit. This is the classic salt-dilution role of cryoprotectants, and it follows directly from the insight in Section 8.
- **Cost:** $\left(w_{\mathrm{gly}}/\mathrm{cap}_{\mathrm{gly}}\right)^2 + \left(w_{\mathrm{sorb}}/\mathrm{cap}_{\mathrm{sorb}}\right)^2$, where each cap is a tolerable concentration taken from the literature.
- **Answer:** the recipe with the lowest cost that meets the target.

How much salt each gram of cryoprotectant removes depends on its molar mass and on the activity model, so the answer now genuinely depends on the thermodynamics: the ideal model and the fitted model recommend different recipes. The B+ experiment then makes the recommended recipe and measures it.

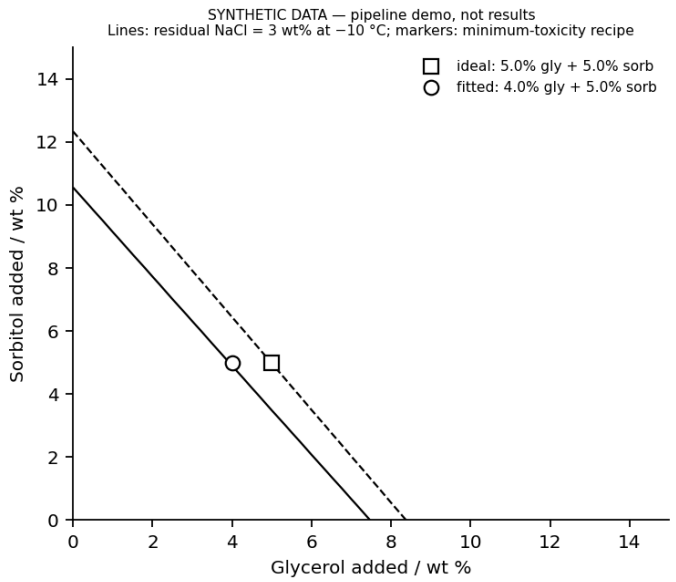


Figure 7: Recipe design on **synthetic** data (pipeline demonstration only). Lines mark recipes that just meet the salt target; markers are the minimum-toxicity choices.

In the code. `thermo/optimise.py` → `design_recipe()`

Takeaway. The recipe is where the model stops describing and starts deciding.

14 Where the model stops working

Every model has edges; stating them is part of the result.

- **Glass transition.** At high cryoprotectant concentration or fast cooling the liquid becomes a glass instead of freezing further. A glass is not at equilibrium, so equation (1) no longer applies — this is exactly the regime vitrification protocols use.
- **Eutectic.** The lowest point where liquid can exist. For these polyols it lies far below what a freezer reaches, so in this project it is a computed prediction, labelled as one.
- **Salt as ideal ions.** NaCl is treated as fully dissociated, ideal ions. Fine at 0.9 wt% saline, approximate at the concentrations freeze concentration produces.
- **No solute–solute interactions.** Tested, not assumed, by the mixture measurements.
- **Environment, not survival.** The model predicts what the cell is exposed to, not whether it survives. That link is biology, and it is contested in the literature.

Takeaway. Knowing where the model fails is as much a result as where it works.

Appendix A — Concept to code map

Concept	Lectures	Code
Freeze concentration	34–35	`sle.residual_liquid`
Equal fugacity, SLE equation	15–17, 25	`sle.freezing_point`, `sle.ln_aw_ice`
Activity coefficients, G^{ex}	21–25	`activity.MargulesModel`
Group contribution	25 (+ UNIFAC idea)	`activity.GroupContributionModel`
Multicomponent mixtures	22–24	`activity._PairModel.ln_gamma_water`
Gibbs–Duhem check	21	`fit.gibbs_duhem_residual`
Osmotic pressure	24	`sle.osmotic_pressure_at_ice`
Fitting, uncertainty	—	`fit.fit_group_contribution`, `fit.bootstrap`
Cooling-curve analysis	—	`analysis.find_freezing_point`
Recipe design	—	`optimise.design_recipe`
Energy balance, Carnot	3, 5, 8	to be added

Appendix B — Glossary

Activity (a_w) Effective "availability" of water in a solution; $a_w = x_w\gamma_w$.

Activity coefficient (γ) Correction from ideal behaviour; $\gamma = 1$ is ideal.

Bootstrap Estimating uncertainty by refitting on resampled copies of the data.

Cryoprotectant Chemical added before freezing to reduce damage.

Eutectic Lowest temperature at which liquid can exist in the mixture.

Excess Gibbs energy The part of mixing energy that an ideal solution does not account for.

Freeze concentration Solutes concentrating in the liquid as pure ice forms.

Fugacity A molecule's tendency to escape a phase; equal in phases at equilibrium.

Gibbs–Duhem Relation linking activity coefficients of all components; consistency test.

Group contribution Assigning interaction parameters to molecular groups, not molecules.

Liquidus Line on a phase diagram where ice first appears.

Recalescence Temperature jump when ice nucleates and releases latent heat.

Supercooling Liquid cooled below its freezing point without yet forming ice.

Vitrification Solidifying into a glass instead of a crystal; not an equilibrium state.